

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA KỸ THUẬT ĐIỆN TỬ I



BÁO CÁO BÀI TẬP LỚN
MÔN: KỸ THUẬT VI XỬ LÝ (ELE1317)

BÀI 2: LẬP TRÌNH HỢP NGỮ ARM
SAO CHÉP VÀ ĐỔI CHỮ HOA – CHỮ THƯỜNG CHUỖI "HELLO WORLD"

Giảng viên hướng dẫn:	
Sinh viên thực hiện:	
Mã sinh viên:	
Lớp / Nhóm:	

Hà Nội, tháng 10/2026

MỤC LỤC

CHƯƠNG 1. ĐỀ BÀI VÀ PHÂN TÍCH.....	4
1.1. Đề bài.....	4
1.2. Phân tích yêu cầu.....	4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	4
2.1. Lỗi ARM Cortex-M3 và tập lệnh Thumb-2.....	4
2.2. Các lệnh sử dụng.....	4
2.3. Nguyên lý đổi chữ hoa – chữ thường.....	5
CHƯƠNG 3. THIẾT KẾ CHƯƠNG TRÌNH.....	5
3.1. Phân bố thanh ghi và bộ nhớ.....	5
3.2. Lưu đồ thuật toán.....	7
3.3. Mã nguồn.....	7
CHƯƠNG 4. CHẠY THỬ VÀ KẾT QUẢ.....	9
4.1. Các bước thực hiện trên Keil μ Vision 5.....	9
4.2. Kết quả mong đợi.....	9
4.3. Kết quả chạy trên Keil.....	9
4.4. Khó khăn gặp phải.....	10
CHƯƠNG 5. KẾT LUẬN.....	10
TÀI LIỆU THAM KHẢO.....	11

DANH MỤC HÌNH ẢNH

Hình 1. Lưu đồ thuật toán chương trình.....	7
Hình 2. Cửa sổ Build Output: "0 Error(s), 0 Warning(s)".....	9
Hình 3. Memory 1 tại 0x77FFFFFF: chuỗi "Hello World".....	9
Hình 4. Memory 1 tại 0x88FFFFFF: bản sao "Hello World".....	10
Hình 5. Memory 1 tại 0x99FFFFFF: chuỗi "HELLO wORLD".....	10

DANH MỤC BẢNG BIỂU

Bảng 1. Các lệnh hợp ngữ sử dụng.....	5
Bảng 2. Ví dụ đổi chữ hoa – chữ thường bằng phép XOR.....	5
Bảng 3. Phân bố thanh ghi.....	6
Bảng 4. Bản đồ bộ nhớ.....	6
Bảng 5. Mã nguồn bai2.s.....	8

Bảng 6. Nội dung bộ nhớ mong đợi sau khi chạy.....	9
Bảng 7. Khó khăn và cách khắc phục.....	10

CHƯƠNG 1. ĐỀ BÀI VÀ PHÂN TÍCH

1.1. Đề bài

Viết chương trình hợp ngữ ARM thực hiện:

- Sao chép chuỗi "Hello World" đang lưu tại địa chỉ 0x77FFFFFF sang địa chỉ 0x88FFFFFF.
- Đổi chữ hoa thành chữ thường và chữ thường thành chữ hoa, lưu chuỗi kết quả ("hELLO wORLD") tại địa chỉ 0x99FFFFFF.

1.2. Phân tích yêu cầu

- Chuỗi gồm 11 ký tự ASCII, kết thúc bằng byte 0 (chuỗi kiểu C), nên dùng ký tự 0 làm điều kiện dừng vòng lặp thay vì đếm cứng độ dài.
- Các địa chỉ 0x77FFFFFF, 0x88FFFFFF, 0x99FFFFFF là địa chỉ lẻ, không chia hết cho 4, nên phải truy cập theo từng byte (LDRB/STRB). Truy cập word tại địa chỉ lẻ sẽ gây lỗi căn lề.
- Ba địa chỉ này không thuộc vùng Flash (0x08000000) hay SRAM (0x20000000) của STM32F103, nên khi chạy trên Keil Simulator phải khai báo cho phép truy cập bằng lệnh MAP (file map_memory.ini).
- Đề chỉ nói chuỗi "đang lưu" tại 0x77FFFFFF, nên chương trình có thêm bước 0: chép chuỗi hằng từ bộ nhớ chương trình vào 0x77FFFFFF để tạo dữ liệu đầu vào.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Lỗi ARM Cortex-M3 và tập lệnh Thumb-2

STM32F103C8T6 dùng lõi ARM Cortex-M3, kiến trúc ARMv7-M, chỉ thực thi tập lệnh Thumb-2 (lệnh 16 và 32 bit). Bộ xử lý có 16 thanh ghi 32 bit: R0–R12 là thanh ghi đa dụng, R13 (SP) là con trỏ ngăn xếp, R14 (LR) là thanh ghi liên kết, R15 (PC) là bộ đếm chương trình; thanh ghi trạng thái xPSR chứa các cờ N, Z, C, V. Cortex-M3 theo kiến trúc load/store: chỉ có lệnh LDR/STR truy cập bộ nhớ, các phép tính đều thực hiện trên thanh ghi.

2.2. Các lệnh sử dụng

Lệnh	Cú pháp ví dụ	Ý nghĩa
LDR (giả lệnh)	LDR R0, =0x77FFFFFF	Nạp hằng 32 bit vào thanh ghi (trình dịch đặt hằng vào literal pool)
LDRB (post-index)	LDRB R2, [R0], #1	Đọc 1 byte tại địa chỉ R0 vào R2, sau đó $R0 = R0 + 1$

Lệnh	Cú pháp ví dụ	Ý nghĩa
STRB (post-index)	STRB R2, [R1], #1	Ghi byte thấp của R2 vào địa chỉ R1, sau đó $R1 = R1 + 1$
CMP	CMP R2, #0x41	Tính $R2 - 0x41$, chỉ cập nhật cờ N, Z, C, V
BEQ / BNE	BNE COPY_LOOP	Rẽ nhánh nếu bằng ($Z = 1$) / khác ($Z = 0$)
BLT / BGT / BLE	BLT CASE_STORE	Rẽ nhánh nếu nhỏ hơn / lớn hơn / nhỏ hơn hoặc bằng (so sánh có dấu)
EOR	EOR R2, R2, #0x20	$R2 = R2 \text{ XOR } 0x20$ (đảo bit 5)
MOVS	MOVS R2, #0	Gán $R2 = 0$ và cập nhật cờ
B	B STOP_HERE	Nhảy không điều kiện (vòng lặp dừng chương trình)

Bảng 1. Các lệnh hợp ngữ sử dụng

Chế độ địa chỉ post-index [Rn], #1 giúp vừa truy cập bộ nhớ vừa tăng con trỏ trong cùng một lệnh, làm vòng lặp ngắn gọn hơn so với dùng thêm lệnh ADD.

2.3. Nguyên lý đổi chữ hoa – chữ thường

Trong bảng mã ASCII, chữ hoa và chữ thường tương ứng chỉ khác nhau ở bit 5 (giá trị 0x20): chữ hoa có bit 5 = 0, chữ thường có bit 5 = 1. Vì vậy chỉ cần một lệnh EOR với 0x20 là đổi được cả hai chiều, không cần hai nhánh cộng/trừ 32 riêng.

Ký tự	Mã hex	Nhị phân	Sau XOR 0x20	Kết quả
H	0x48	0100 1000	0110 1000 = 0x68	h
e	0x65	0110 0101	0100 0101 = 0x45	E
W	0x57	0101 0111	0111 0111 = 0x77	w
(dấu cách)	0x20	0010 0000	không đổi (không phải chữ cái)	(dấu cách)

Bảng 2. Ví dụ đổi chữ hoa – chữ thường bằng phép XOR

Trước khi đảo bit, chương trình phải kiểm tra ký tự có phải chữ cái không (nằm trong 0x41–0x5A hoặc 0x61–0x7A). Nếu không kiểm tra, dấu cách (0x20) sẽ bị đổi thành 0x00 và làm chuỗi kết thúc sớm.

CHƯƠNG 3. THIẾT KẾ CHƯƠNG TRÌNH

3.1. Phân bổ thanh ghi và bộ nhớ

Thanh ghi	Vai trò
R0	Con trỏ nguồn (địa chỉ đọc)

Thanh ghi	Vai trò
R1	Con trỏ đích (địa chỉ ghi)
R2	Byte (ký tự) đang xử lý

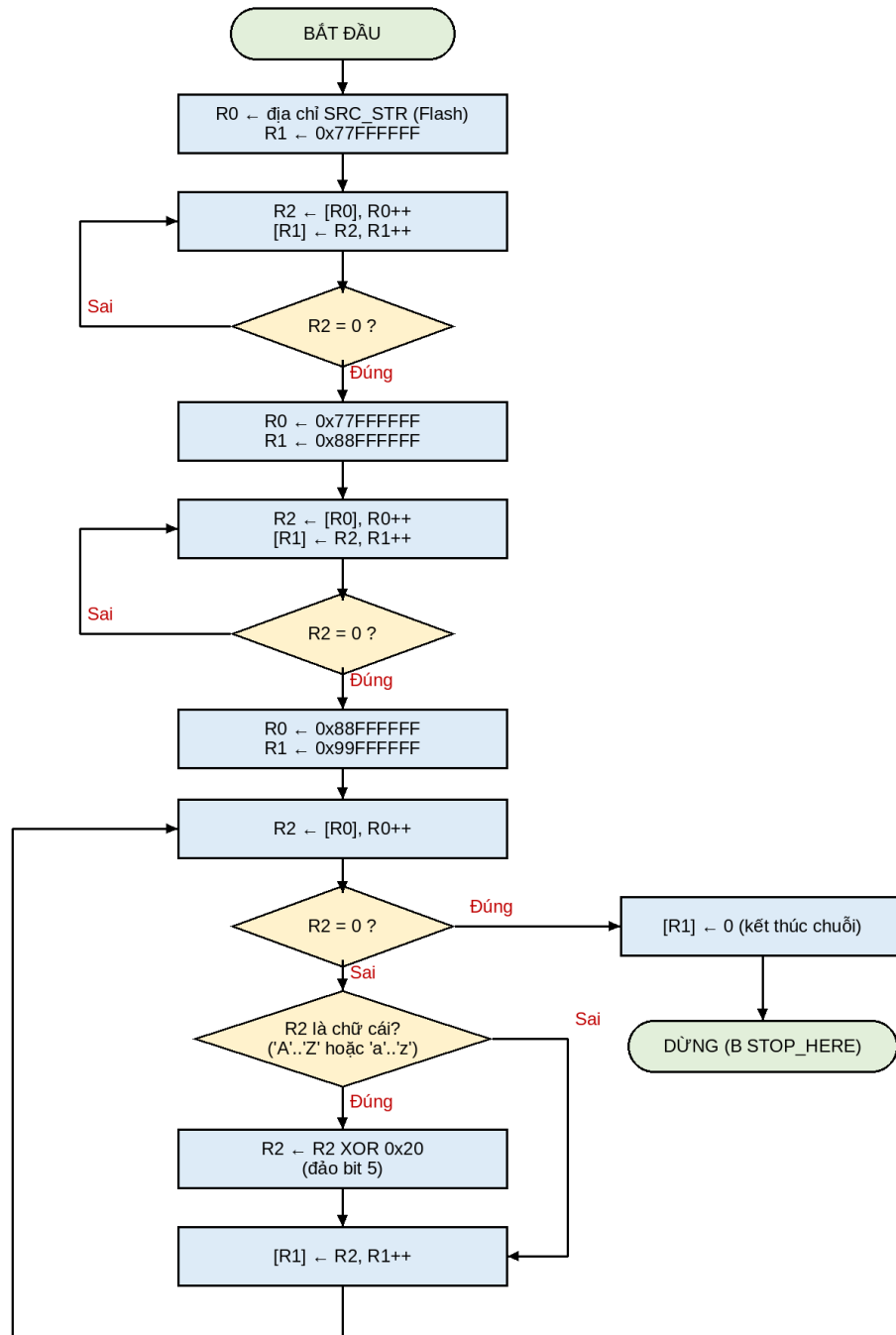
Bảng 3. Phân bố thanh ghi

Vùng nhớ	Địa chỉ	Nội dung
SRC_STR	Flash (literal trong chương trình)	"Hello World", 0
Nguồn	0x77FFFFFF – 0x7800000A	"Hello World", 0
Bản sao	0x88FFFFFF – 0x8900000A	"Hello World", 0
Kết quả	0x99FFFFFF – 0x9A00000A	"hELLO wORLD", 0

Bảng 4. Bản đồ bộ nhớ

Mỗi chuỗi chiếm 12 byte (11 ký tự và byte 0), nên vượt qua ranh giới 0x..FFFFFF sang vùng 0x..000000. Vì vậy file map_memory.ini khai báo mỗi vùng từ 0x..FFFF00 đến 0x..0000FF.

3.2. Lưu đồ thuật toán



Hình 1. Lưu đồ thuật toán chương trình

3.3. Mã nguồn

Chương trình viết cho trình dịch armasm (ARM Compiler 5) trong Keil μ Vision 5, dùng file khởi động chuẩn của Keil; hàm main được viết bằng hợp ngữ.

```

; =====
; Bai 2 - Hop ngu ARM (Cortex-M3)
; "Hello World" luu tai 0x77FFFFFF -> copy sang 0x88FFFFFF
; -> doi hoa/thuong ("HELLO wORLD") luu vao 0x99FFFFFF
; Dung voi startup chuan cua Keil (Device -> Startup), ham main viet bang ASM
; =====
                PRESERVE8
                THUMB
                AREA    |.text|, CODE, READONLY

main            PROC
EXPORT main

; ---- B0: dat chuoi "Hello World" vao dia chi 0x77FFFFFF ----
                LDR     R0, =SRC_STR
                LDR     R1, =0x77FFFFFF
INIT_LOOP      LDRB    R2, [R0], #1
                STRB    R2, [R1], #1
                CMP     R2, #0
                BNE     INIT_LOOP

; ---- B1: copy 0x77FFFFFF -> 0x88FFFFFF ----
                LDR     R0, =0x77FFFFFF
                LDR     R1, =0x88FFFFFF
COPY_LOOP      LDRB    R2, [R0], #1
                STRB    R2, [R1], #1
                CMP     R2, #0                ; gap '\0' thi dung
                BNE     COPY_LOOP

; ---- B2: doi hoa <-> thuong, luu vao 0x99FFFFFF ----
                LDR     R0, =0x88FFFFFF
                LDR     R1, =0x99FFFFFF
CASE_LOOP      LDRB    R2, [R0], #1
                CMP     R2, #0
                BEQ     CASE_DONE
                CMP     R2, #0x41            ; 'A'
                BLT     CASE_STORE
                CMP     R2, #0x5A            ; 'Z'
                BLE     CASE_FLIP
                CMP     R2, #0x61            ; 'a'
                BLT     CASE_STORE
                CMP     R2, #0x7A            ; 'z'
                BGT     CASE_STORE
CASE_FLIP      EOR     R2, R2, #0x20        ; dao bit 5
CASE_STORE     STRB    R2, [R1], #1
                B       CASE_LOOP
CASE_DONE      MOVS    R2, #0
                STRB    R2, [R1]            ; ket thuc chuoi

STOP_HERE     B       STOP_HERE            ; dung tai day, mo Memory de xem
                ENDP

SRC_STR        DCB     "Hello World", 0
                ALIGN
                END

```

Bảng 5. Mã nguồn bai2.s

File map_memory.ini (khai báo vùng nhớ cho Simulator):

```

// Cho phép Simulator truy cập 3 vùng nhớ của đề bài
MAP 0x77FFF00, 0x780000FF READ WRITE
MAP 0x88FFF00, 0x890000FF READ WRITE
MAP 0x99FFF00, 0x9A0000FF READ WRITE

```


CHƯƠNG 4. CHẠY THỬ VÀ KẾT QUẢ

4.1. Các bước thực hiện trên Keil μ Vision 5

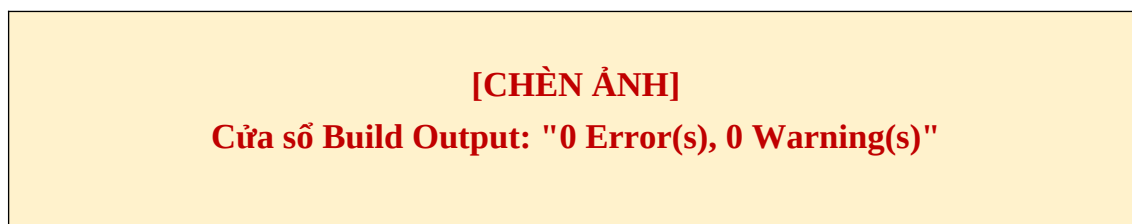
- Project → New μ Vision Project, chọn thiết bị STM32F103C8. Lưu project ở đường dẫn không dấu, không khoảng trắng.
- Trong Manage Run-Time Environment chọn CMSIS → CORE và Device → Startup.
- Thêm file bai2.s vào Source Group 1.
- Options for Target → Debug: chọn Use Simulator, mục Initialization File chọn map_memory.ini.
- Build (F7): kết quả 0 Error(s).
- Debug → Start/Stop Debug Session (Ctrl+F5), chạy (F5) rồi dừng (Stop) khi chương trình đã tới STOP_HERE.
- Mở View → Memory Windows → Memory 1, lần lượt nhập 0x77FFFFFF, 0x88FFFFFF, 0x99FFFFFF để xem kết quả.

4.2. Kết quả mong đợi

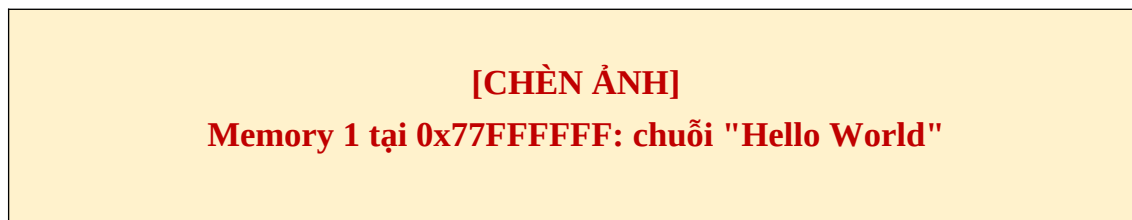
Địa chỉ	Nội dung (hex)	Chuỗi
0x77FFFFFF	48 65 6C 6C 6F 20 57 6F 72 6C 64 00	Hello World
0x88FFFFFF	48 65 6C 6C 6F 20 57 6F 72 6C 64 00	Hello World
0x99FFFFFF	68 45 4C 4C 4F 20 77 4F 52 4C 44 00	hELLO wORLD

Bảng 6. Nội dung bộ nhớ mong đợi sau khi chạy

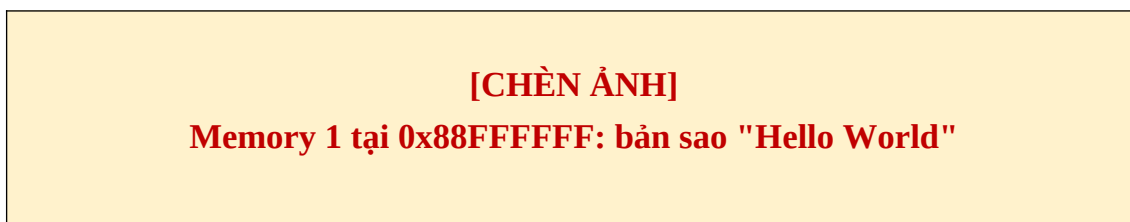
4.3. Kết quả chạy trên Keil



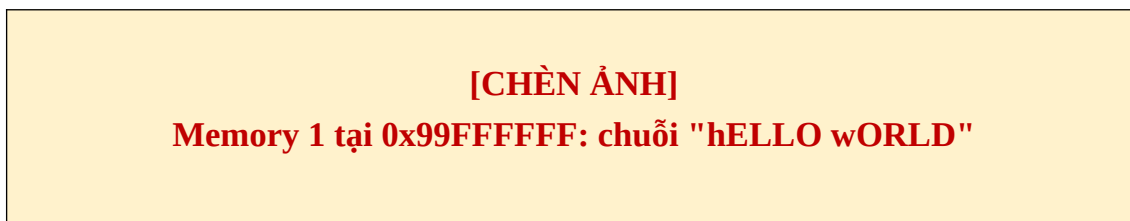
Hình 2. Cửa sổ Build Output: "0 Error(s), 0 Warning(s)"



Hình 3. Memory 1 tại 0x77FFFFFF: chuỗi "Hello World"



Hình 4. Memory 1 tại 0x88FFFFFF: bản sao "Hello World"



Hình 5. Memory 1 tại 0x99FFFFFF: chuỗi "hELLO wORLD"

So sánh với bảng kết quả mong đợi cho thấy chương trình sao chép đúng chuỗi và đổi đúng chữ hoa – chữ thường; dấu cách (0x20) và ký tự kết thúc (0x00) được giữ nguyên.

4.4. Khó khăn gặp phải

Hiện tượng	Nguyên nhân	Cách khắc phục
Simulator báo lỗi "no 'write' permission" khi ghi vào 0x77FFFFFF	Địa chỉ không thuộc vùng nhớ của STM32F103	Khai báo MAP ... READ WRITE trong map_memory.ini và đặt làm Initialization File
Keil không mở được project, báo lỗi đường dẫn	Đường dẫn thư mục có dấu tiếng Việt, khoảng trắng	Đặt project ở thư mục không dấu, ví dụ D:\KTVXL\Bai2
Dấu cách bị biến mất nếu đảo bit mọi ký tự	$0x20 \text{ XOR } 0x20 = 0x00$ (ký tự kết thúc chuỗi)	Chỉ đảo bit 5 với ký tự là chữ cái

Bảng 7. Khó khăn và cách khắc phục

CHƯƠNG 5. KẾT LUẬN

- Chương trình đáp ứng yêu cầu: sao chép chuỗi từ 0x77FFFFFF sang 0x88FFFFFF và lưu chuỗi đã đổi chữ hoa – chữ thường tại 0x99FFFFFF.
- Hiểu được kiến trúc load/store của Cortex-M3, các lệnh LDRB/STRB với chế độ post-index, so sánh và rẽ nhánh có điều kiện, phép logic EOR.
- Hiểu cách dùng Keil Simulator: bản đồ bộ nhớ, lệnh MAP và cửa sổ Memory để kiểm tra kết quả.
- Hướng mở rộng: viết thành chương trình con nhận địa chỉ nguồn/đích qua R0, R1 (theo chuẩn AAPCS) để tái sử dụng cho chuỗi bất kỳ.

TÀI LIỆU THAM KHẢO

- [1] ARM, ARMv7-M Architecture Reference Manual.
- [2] ARM, Cortex-M3 Technical Reference Manual.
- [3] ARM, ARM Compiler armasm User Guide (Keil MDK 5).
- [4] STMicroelectronics, PM0056 – STM32F10xxx Cortex-M3 Programming Manual.
- [5] Bài giảng Kỹ thuật vi xử lý, Học viện Công nghệ Bưu chính Viễn thông.